Projet Structure de Données : Resilience Road Network

Loïc Lecharlier et José Vander Meulen

Table des matières

1 Contexte et Objectifs **1**

2 Modélisation Physique **1**

3 Modélisation Algorithmique **2**

4 Phase 1 : Analyse Statique **2**
4.1 Algorithme 1 : Simulation de la Crue 3
4.2 Algorithme 2 : Navigation de Secours 3

5 Phase 2 : Analyse Temporelle **4**
5.1 Algorithme 3 : Chronologie de la Crue 4
5.2 Algorithme 4 : Évacuation Dynamique 5

6 Délivrables **6**

7 Critères de Validation **6**

8 Classes Tests **7**

1 Contexte et Objectifs

La ville de Bruxelles aimerait avoir un simulateur de crise pour le cas où elle serait menacée par une inondation majeure.
Ce projet vise donc à développer le noyau algorithmique d'un tel simulateur.
Le système doit gérer des graphes de grande taille (**1000000 de noeuds**) et répondre à deux impératifs :

1) **Cartographier le danger :** Déterminer l'emprise et la vitesse de l'inondation.
2) **Sauver des vies :** Calculer des itinéraires d'évacuation dans une "course contre la montre".

2 Modélisation Physique

Le comportement de l'eau est régi par la topographie.
Il y a donc deux règles physiques qui vont conditionner vos algorithmes :

A) **La Pente (S)**
Pour chaque segment reliant un noeud A à un voisin B, la pente détermine la vitesse de l'eau :

$$S = \frac{Altitude(A) - Altitude(B)}{Distance(A, B)}$$

B) **Condition de Propagation (Tolérance ϵ)**
L'eau se propage d'un noeud inondé X vers un voisin Y si le relief le permet.
De plus, une tolérance ϵ (en mètres) est introduite pour simuler l'accumulation ou le débordement :

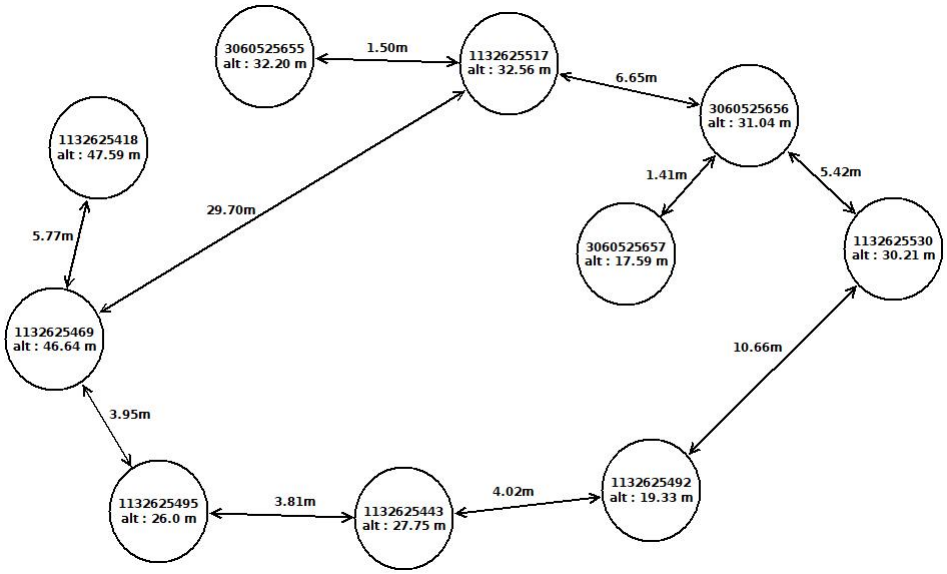
$$Altitude(Y) \leq Altitude(X) + \epsilon$$

3 Modélisation Algorithmique

Dans notre implémentation, nous allons représenter la ville de Bruxelles par un graphe définit comme suit :

- Un noeud représente un point géographique dans la ville et a 5 caractéristiques
  - un id,
  - une latitude,
  - une longitude,
  - un nom,
  - une altitude (en mètre).
- Un arc réprésente une rue reliant deux points géographiques et a 4 caractéristiques
  - le point d'origine,
  - le point d'arrivée,
  - la distance entre le noeud de départ et le noeud d'arrivée,
  - le nom de la rue.

  Voici le graphe (basé sur les fichiers nodes_10.csv et edges_10.csv) qui va nous service pour illustrer les algorithmes à implémenter :



4 Phase 1 : Analyse Statique

Dans cette première phase, on ne va pas tenir compte du temps.
Les objectifs de cette phase sont

1) Déterminer la zone qui sera inondée sur base des points de départ de l'eau.
2) Connaissant la zone inondée, trouvez le chemin le plus court, en nombre de rues, pour aller d'un point de départ à un point sur en contournant la zone inondée.

4.1 Algorithme 1 : Simulation de la Crue

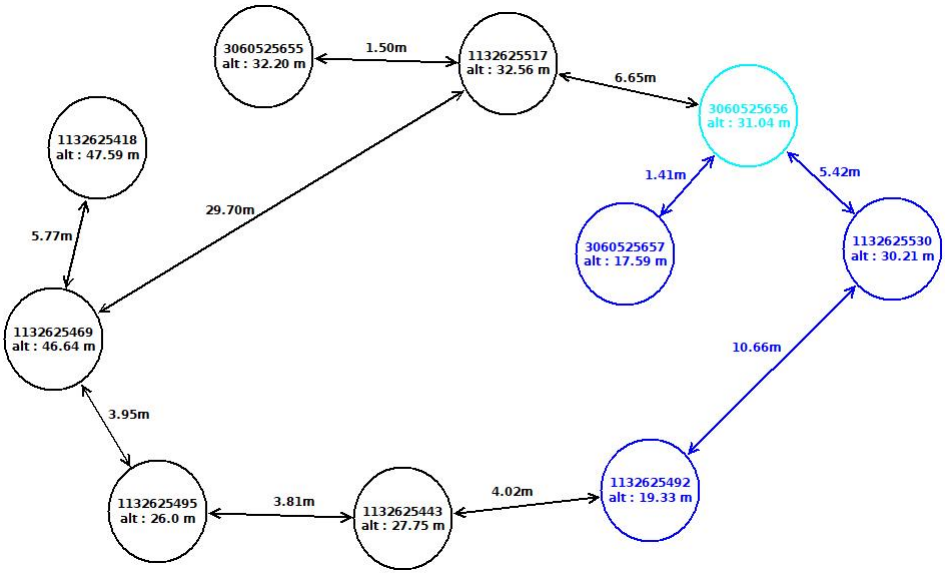
L'objectif de cet algorithme est de déterminer la zone d'inondation maximale.
Pour cela, on va considérer que l'eau passe d'un point géographique à un autre si

1) il y a une rue menant du premier point géographique au second,
2) la condition de la section 2 est vérifiée.

La méthode qui va implémenter cet algorithme

- recevra les ids des points de départ comme paramètre,
- va parcourir tous le graphe selon la logique défine à la section 2
- va renvoyer un tableau contenant les identifiants de tous les noeuds qui représente une point géographique inondé (qui a été atteint par l'eau) dans l'ordre où ils ont été parcourus par l'eau.

Si dans l'exemple du graphe, l'inondation commence au point 3060525656, alors la zone inondée est celle en bleu :

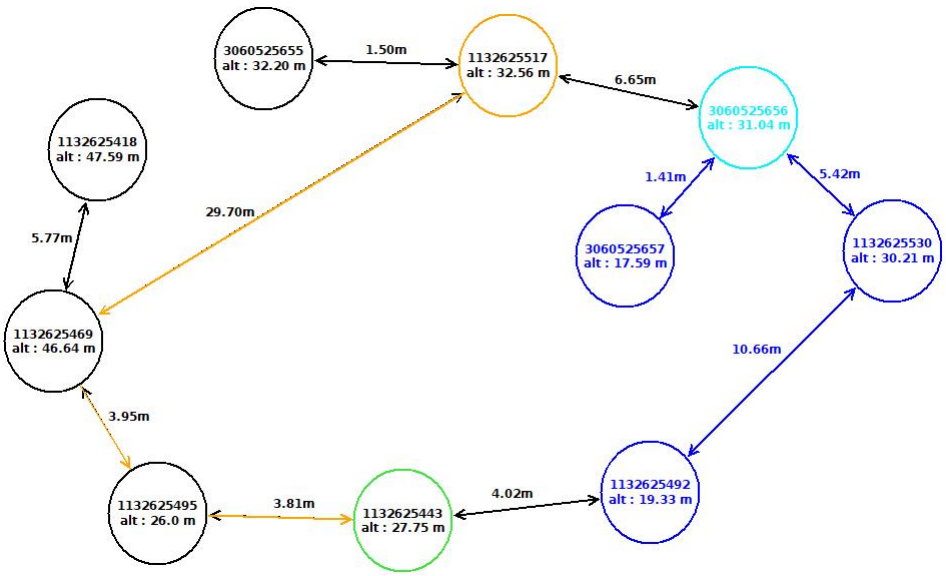


4.2 Algorithme 2 : Navigation de Secours

L'objectif est de trouver le chemin le plus court en "nombre de rues" pour contourner la zone inondée.
La méthode qui va implémenter cela

- recevra en paramètre :
  - le point géographique de départ
  - le point géographique d'arrivée
  - le tableau des points géographique renvoyé par l'algorithme 1
- renverra une liste contenant le chemin de plus court, en nombre de rues, du point de départ au point d'arrivée sans passer par un point inondé.

Si dans l'exemple du graphe, on a la zone inondée de la zone précédente et qu'on veut aller du point 1132625517 au point 1132625443 alors le chemin le plus court est celui en orange :



On voit bien que l'eau ne fait que descendre.

5 Phase 2 : Analyse Temporelle

Cette phase introduit la dimension temporelle (le temps t).
En effet, lors d'une inondation l'eau avance à une certaine vitesse et tous les points ne deviennent pas inondé "en même temps".
Les objectifs de cette phase sont donc

1) Déterminer le moment où chaque point est susceptible d'être inondé
2) Connaissant la chronologie de l'inondation, déterminer le chemin le plus rapide d'un point géographique à un autre sans croiser l'eau.

5.1 Algorithme 3 : Chronologie de la Crue

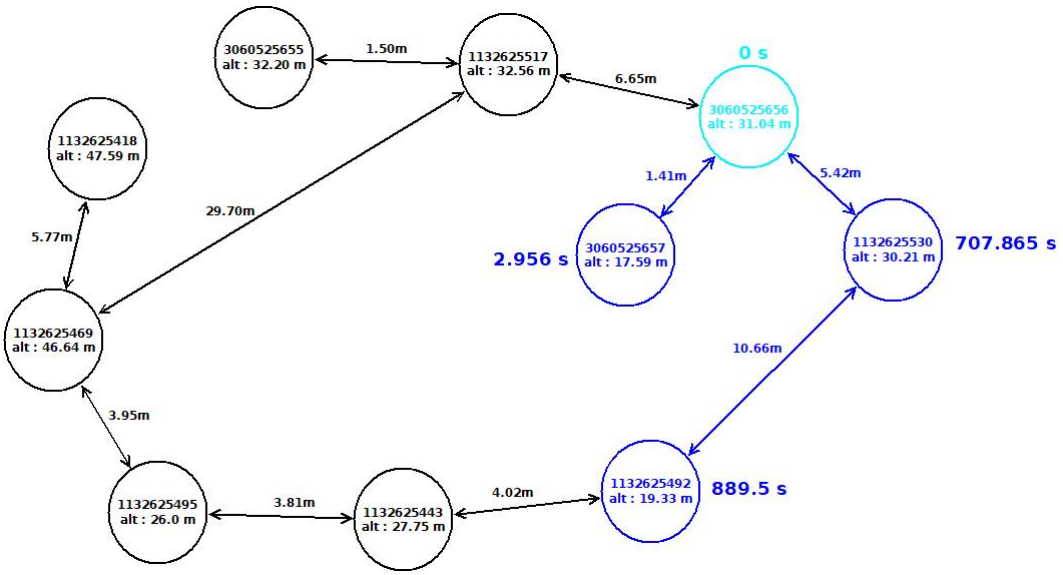
L'objectif est de calculer l'instant précis t_{flood} où l'eau atteint chaque noeud, sachant que

- l'eau prend le chemin le plus rapide (qui prend le moins de temps)
- l'eau suit les rues
- le point géographique suivant atteint par l'eau dépend de la vitesse de l'eau et de la distance entre les points.
  Or la vitesse de l'eau dépend de la pente de la rue S (voir section 2).
  La vitesse de l'eau départ de l'inondation sera donnée par V_{water} et l'eau va accélérer en descendant de point en point.
  Donc si on passse d'un point p_1 à un point p_2 on va avoir $V_{water}(p_2) = V_{water}(p_1) + (k \cdot S)$ en prenant $k = 0.05$.
  Pour tenir compte de cette vitesse et de la distance entre deux points le poids p d'un arc entre 2 points sera donné par $p = \dfrac{Distance\ entre\ les\ 2\ points}{V_{water}}$ (le temps de parcours de la distance représentée par l'arc).
- Attention la seule condition de propagation est que si la vitesse devient négative ou nulle alors l'eau s'arrête et ne suis pas l'arc.

La méthode qui implémentera cet algorithme

- aura les ids des points de départs et la vitesse initiale de l'eau comme paramètre,
- renverra un tableau t_{food} tel que $t_{food}[i]$ est le temps (en secondes) à partir du quel le noeud i est inondé.

Si dans l'exemple du graphe, l'inondation commence au point 3060525656, alors la chronologie de l'inondation est celle en bleu :



5.2 Algorithme 4 : Évacuation Dynamique

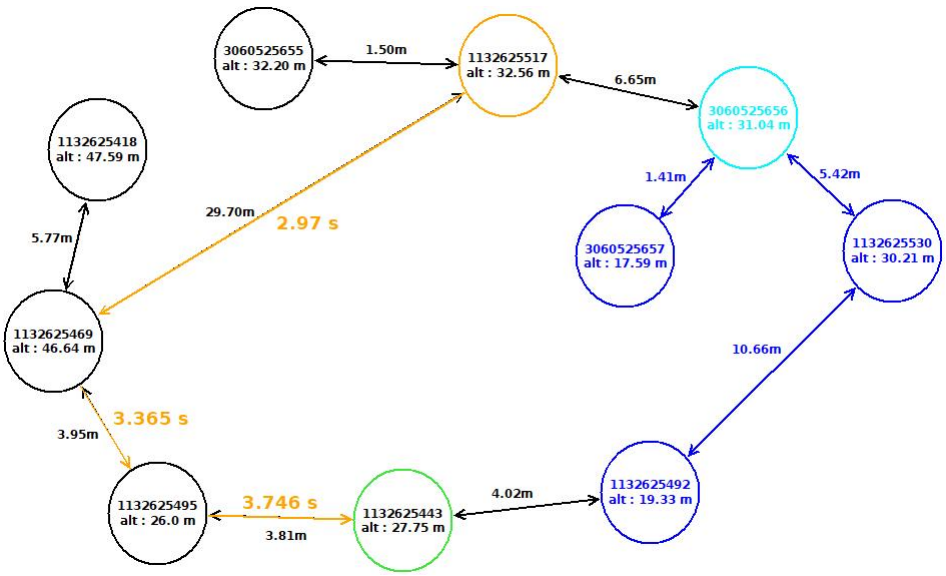
L'objectif est de trouver l'itinéraire le plus rapide pour un véhicule, en tenant compte du fait que des routes se ferment au cours du temps car elles sont inondées.
Pour facilité, on va considérer que la vitesse du véhicule est constante et donnée par $V_{vehicle}$.
Alors le poids p d'un arc entre points géographique de sera donné par

$$t = \frac{Distance\ entre\ les\ 2\ points}{V_{vehicule}}.$$

La méthode qui va implémenter cette algorithme

- aura comme paramètres
  - le point de départ,
  - le point d'arrivée,
  - la vitesse du véhicule,
  - le tableau t_{flood} généré par l'algorithme 3,
- va considérer qu'au départ $t = 0$.
- renverra une liste de noeud représenant le chemin le plus cours (en terme poids des arcs) entre le point de départ et le point d'arrivée en considérant que
  - après avoir parcouru en arc t devient $t = t + \frac{distance\,entre\,les\,2\,points}{V_{vehicule}}$
  - on ne peut pas choisir un arc si le point d'arrivée est inondé autrement dit si $t > t_{flood}[i]$ où t est le temps à l'arrivée sur le point d'arrivée et i est l'id du point d'arrivée.

Si dans l'exemple du graphe, on a la zone inondée de la zone précédente et qu'on veut aller du point 1132625517 au point 1132625443 alors le chemin le plus rapide est celui en orange avec les temps indiqués :



6 Délivrables

Pour le dimanche 29 mars à 20h00 vous devrez soumettre une archive zip contenant les classes que vous aurez créées pour votre implémentations.
Parmi celles-ci il y doit y avoir :

- une classe implémentant un noeud du graphe,
- une classe implémentant un arc du graphe,
- une classe implémentant le graphe avec les méthodes demandées. Les en-têtes de ces méthodes sont
  - `public Localisation[] determinerZoneInondee(long[] idsDepart, double epsilon)`
  - `public Deque<Localisation> trouverCheminLePlusCourtPourContournerLaZoneInondee(long idDepart, long idArrivee, Localisation[] zoneInondee)`
  - `public Map<Localisation,Double> determinerChronologieDeLaCrue(long[] idsDepart, double vWaterInit, double k)`
  - `public Deque<Localisation> trouverCheminDEvacuationLePlusCourt(long idDepart, long idEvacuation, double vVehicule, Map<Localisation,Double> tFlood)`

  Vous devez respecter scrupuleusement ces en-têtes car seront celles que nous utiliserons dans notre classe test ainsi que dans celle que nous vous fournirons.

7 Critères de Validation

Voici les différents critères qui guiderons nos corrections :

1) **Respect des contraintes physiques :** L'eau ne remonte pas les pentes fortes (sauf tolérance ϵ) et l'eau n'avance que si sa vitesse est positive.
2) **Respect des contraintes temporelles :** Le véhicule n'arrive jamais sur un noeud après ou au moment où celui-ci est inondé.
3) **Complexité et Mémoire :**
4) Les structures de données doivent permettre de traiter **1000000 de noeuds**.
   - $\Longrightarrow$ Le choix des structures de données pour implémenter le graphe doit optimiser l'espace utilisé.
   - $\Longrightarrow$ Les algorithmes utilisés doivent avoir une complexité temporelle optimale.
5) **Maintenabilité du code :** Code propre, lisible et non redondant.

8 Classes Tests

Pour tester vos méthodes, nous vous fournissons 5 classes tests :

- `TestSimulator10.java` : graphes avec 10 noeuds et une dizaine d'arcs ;
- `TestSimulator1000a.java` : graphes avec 1000 noeuds et un millier d'arcs ;
- `TestSimulator1000b.java` : graphes avec 1000 noeuds et un millier d'arcs :
  ⇒ Ville presque totalement inondée et chemin d'évacuation le plus rapide plus court que le plus court chemin évitant la zone inondée ;
- `TestSimulator1000000a.java` : graphes avec 1000000 noeuds et un million d'arcs : plusieurs points de départ pour l'inondation ;
- `TestSimulator1000000b.java` : graphes avec 1000000 noeuds et un million d'arcs : plusieurs points de départ pour l'inondation
  ⇒ Chemins très longs et chemin d'évacuation le plus rapide plus long que le plus court chemin évitant la zone inondée.

Les résultats à obtenir ainsi qu'un ordre d'idée du temps d'exécution se trouvent respectivement dans les fichiers

- `TestSimulator10_Resultats.java` ;
- `TestSimulator1000a_Resultats.java` ;
- `TestSimulator1000b_Resultats.java` ;
- `TestSimulator1000000a_Resultats.java` ;
- `TestSimulator1000000b_Resultats.java`.